

## Two ways to separate strings

So far, the output has been printed on one line with a space between the items. Here are two other ways of separating strings.

```
>>> print(a, b, c, sep='-')
Dave-is-12
```

### △ Hyphenate the outputs

A hyphen can be put between the variables when they're printed. Other characters, such as "+" or "\*", can be used too.

The character between the outputs

```
>>> print(a, b, c, sep='\n')
Dave
is
12
```

The separator

Each variable starts on a new line

### △ Outputs on new lines

The space or character between the outputs is called a "separator" ("sep"). Using "\n" prints each output on a new line.

## Three ways to end output

There are several different ways you can signal the end of the output of a "print" function.

```
>>> print(a, '.')
Dave .
```

Period added as a string

```
>>> print(a, end='.')
Dave.
```

Period added as an "end" character

### △ Add a period to the output

A period can be added as another string to be printed, but it will print with a space before it. To avoid this, use "end= '.'" instead.

```
>>> for n in range(3):
    print('Hurray!' end=' ')
Hurray! Hurray! Hurray!
```

Loop to print three times

Space as "end" character

Output is all printed on one line

### △ Output on one line

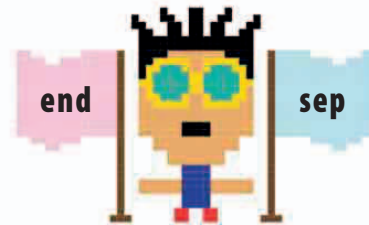
Usually, each new "print" command starts on a new line. To get the output all on one line use a space as the "end" character.



### EXPERT TIPS

## Options at the end

The "end" and "sep" labels tell Python that the next item in the program isn't just another string. Remember to use them; otherwise, the program will not work correctly.



```
>>> print(a, end='\n\n\n\n')
Dave
```

Each "\n" starts a new line

Blank space before the prompt

```
>>>
```

### △ Blank lines at the end

Using "\n" starts each output from a new line. Several of them can be used together to add blank lines at the end of a program.